



Bachelorarbeit

Entwicklung eines GDB Stubs in Rust für D3OS

vorgelegt von

Leon Palikuca

aus Trier

Abteilung Betriebssysteme
Prof. Dr. Michael Schöttner
Heinrich-Heine-Universität Düsseldorf

4. August 2026

Erstgutachter: Prof. Dr. Michael Schöttner
Zweitgutachter: Prof. Dr. Stefan Conrad
Betreuer: Fabian Ruhland, Niklas Sombert

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielsetzung	1
1.3	Verwendete Tools und Entwicklungsumgebung	2
2	Grundlagen	3
2.1	Rust	3
2.2	D3OS	3
2.3	GDB	3
2.4	Remote Debugging	4
2.5	Verwandte Projekte	4
2.6	Umsetzung von GDB Befehlen über das Remote Serial Protocol	4
2.6.1	Lesen & Schreiben von Speicher	4
2.6.2	Continue von einem Breakpoint	4
2.6.3	Step Into	5
2.6.4	Step Over	5
3	Implementierung	7
3.1	Connection	7
3.2	BlockingEventLoop	7
3.2.1	wait_for_stop_reason	8
3.2.2	on_interrupt()	8
3.3	DebugState	8
3.4	Target	8
3.4.1	Inlineable Dyn Extension Traits	9
3.4.2	read_registers	9
3.4.3	write_registers	10
3.4.4	read_addrs	11
3.4.5	write_addrs	11
3.4.6	list_active_threads	11
3.5	Software Breakpoints	11
3.5.1	add_sw_breakpoint	11
3.5.2	remove_sw_breakpoint	12
3.5.3	Breakpoint Handler	12
3.6	MultiThreadResume	16
3.6.1	clear_resume_actions	16
3.6.2	set_resume_action_continue	16
3.6.3	set_resume_action_step	16

3.6.4	set_resume_action_scheduler_lock	16
3.6.5	Resume	16
3.6.6	Debug Exception Handler	17
3.7	Hardware Breakpoints	17
3.7.1	add_hw_breakpoint	18
3.7.2	remove_hw_breakpoint	18
3.7.3	Debug Exception Handler Erweiterung	18
4	Benutzung	20
4.1	Kompilieren	20
4.2	Verwendung im Terminal	20
4.3	Verwendung mit VS Code	21
5	Evaluation	23
5.1	Breakpoints	23
5.2	Single-Stepping	23
5.3	Registerzugriff	24
5.4	Speicher auslesen	24
5.5	Probleme des Serial Ports	24
5.6	Grenzen der Implementierung	25
6	Fazit und Ausblick	27
6.1	Fazit	27
6.2	Ausblick	27
	Literatur	29
	Abbildungsverzeichnis	30

Kapitel 1

Einleitung

Ein Debugger ist ein grundlegendes Werkzeug zur Entwicklung komplexer Systeme und Programme. Er ermöglicht es unter anderem, Programme anzuhalten und Speicher auszulesen. Für Anwendungen, die auf etablierten Betriebssystemen laufen, stehen Debuggingwerkzeuge, wie der GDB (GNU Debugger), zur Verfügung. Es kann aber sein, dass auf einem System kein vollständiger Debugger ausgeführt werden kann. Dazu zählen beispielsweise Betriebssystemkernel oder Betriebssysteme, die noch nicht alle Funktionen unterstützen, die ein Debugger benötigt. In solchen Fällen kann Remote-Debugging verwendet werden, um dennoch das System auf Fehler untersuchen zu können [1].

1.1 Motivation

D3OS ist ein Betriebssystem, das von der Abteilung für Betriebssysteme der HHU entwickelt wird. Es kann aktuell mithilfe von QEMU und GDB debuggt werden [2]. QEMU ist ein Open-Source Systememulator, beziehungsweise Virtualisierungsprogramm. Es stellt eine Debugging-Schnittstelle bereit, über die GDB zum Beispiel den emulierten Systemzustand kontrollieren, Speicher auslesen und Breakpoints setzen kann [3]. Dadurch kann D3OS in einer emulierten Umgebung auf Fehler untersucht werden.

Die Ausführung in einem Emulator unterscheidet sich jedoch von der Ausführung auf echter Hardware. QEMU abstrahiert und emuliert Hardwareverhalten. Es kann aber sein, dass bestimmte Fehler von zum Beispiel Hardwareeigenschaften oder Timings abhängig sind. Manche Fehler könnten also durch die emulierte Hardware verhindert werden, aber auf echter Hardware auftreten. In solchen Fällen reicht der QEMU-Debugger nicht.

D3OS soll nicht nur in QEMU, sondern auch auf realer Hardware auf Fehler untersucht werden können. Da auf dem Zielsystem kein vollständiger Debugger ausgeführt werden kann, muss der Kernel eine Schnittstelle bereitstellen, über die GDB mit ihm kommunizieren kann. Das kann durch einen GDB-Remote-Stub erreicht werden, der als Kernel-Thread von D3OS läuft und dadurch Remote-Debugging ermöglicht.

1.2 Zielsetzung

Ziel dieser Arbeit ist die Entwicklung und Integration eines GDB-Stubs für D3OS. Dieser soll es ermöglichen, das Betriebssystem und darauf ausgeführten Code auf echter Hardware zu debuggen. Der Stub kommuniziert über das GDB Remote Serial Protocol mit einer

GDB Instanz, die auf einem anderen Host-Rechner ausgeführt wird. Als Grundlage für die Kommunikation wird die Rust Crate `gdbstub` verwendet, die von GDB empfangene Pakete verarbeitet und entsprechende Funktionen aufruft [4]. Die Pakete werden über den seriellen Port ausgetauscht.

Der Stub soll im Kernel laufen und grundlegende Debugging-Funktionen bereitstellen. Dazu gehören das Lesen und Schreiben von Registern, Zugriff auf Speicher, das Setzen und Entfernen von Breakpoints sowie die Unterstützung von Single-Stepping.

1.3 Verwendete Tools und Entwicklungsumgebung

Der Stub wurde in einer Kopie des Repository von D3OS entwickelt [2], welches ebenfalls stetig weiterentwickelt wurde. Während der Entwicklung des Stubs wurde der Scheduler von D3OS von einem Single-Core-Scheduler zu einem Multi-Core-Scheduler geändert. Da der Stub die Kontrolle über alle laufenden Threads braucht, ist er auf Funktionen des Schedulers angewiesen, die sich mit der Umstellung anders verhalten. Auf der neusten Version des Stubs funktioniert dieser mit dem neuen Scheduler in QEMU, aber nicht auf echter Hardware. Die Probleme die dabei auftreten werden im Kapitel Evaluation beschrieben. Die nutzbare Version des Stubs basiert auf einer älteren Version von D3OS.

Als Entwicklungsumgebung wurde Visual Studio Code [5] ohne Add-Ons verwendet. Zum testen der graphischen Benutzeroberfläche von VS Code wurden die entsprechenden Extensions zur Nutzung von GDB in VS Code verwendet. Um möglichst schnell und effizient neue Funktionen zu testen, wurde QEMU verwendet. Der Stub wurde aber auch auf echter Hardware auf den Bildungsrechnern der Universität getestet.

ChatGPT [6] wurde zur Fehlersuche und als Suchmaschine während dieser Arbeit verwendet.

Kapitel 2

Grundlagen

In dieser Arbeit werden grundlegende Kenntnisse in der Entwicklung von Betriebssystem und der Verwendung eines Debuggers vorausgesetzt. Bestimmte Begriffe und Konzepte werden daher nachfolgend ohne ausführliche Erklärung verwendet. Die wichtigsten Grundlagen zum Verständnis dieser Arbeit werden in diesem Kapitel jedoch kurz erläutert.

2.1 Rust

Rust kombiniert die Möglichkeit, hardwarenahe Details, wie Speicherzugriffe, selber zu kontrollieren, was normalerweise mit viel Aufwand verbunden ist, mit einer hohen Nutzerfreundlichkeit. Es wird viel Wert sowohl auf Geschwindigkeit, als auch auf Stabilität gelegt. Der Compiler schließt viele Bugs bereits zur Compilezeit aus, was späteres Testen und Suchen verringert [7].

2.2 D3OS

Das D3OS (Distributed Data-Driven OS) ist ein in Rust entwickeltes, verteiltes Betriebssystem, das speziell für moderne Rechenzentrenhardware und skalierbare, datenintensive Anwendungen entworfen wurde. Es setzt auf verteilte Ressourcenverwaltung, einen Shared-nothing Microkernel, sowie schnelle Netzwerktechnologien [8]. Der Quellcode zu D3OS ist auf GitHub zu finden und wird von der Abteilung für Betriebssysteme der HHU betreut und entwickelt [2].

Bei einem Microkernel handelt es sich um eine Architektur, bei welcher der Kernel nur grundlegende Dienste, wie zum Beispiel Allokation von physischem Speicher und Scheduling übernimmt, während andere Komponenten, wie das Dateisystem, modular als Server im User-Space realisiert werden. Dadurch sind Microkernels zwar stabiler und flexibler, da ein fehlerhafter Dienst nicht direkt den Kernel beeinflusst, aber auch langsamer als monolithische Kernels, da mehr Prozesse miteinander kommunizieren müssen und mehr Kontextwechsel stattfinden [9].

2.3 GDB

Bei GDB handelt es sich um einen weit verbreiteten Open-Source Debugger. GDB unterstützt viele verschiedene Sprachen, darunter auch das in D3OS verwendete Rust und

kann sowohl lokal, als auch remote auf einem anderen Zielsystem verwendet werden [1].

2.4 Remote Debugging

Wie bereits erwähnt bietet GDB die Möglichkeit auf einem Hostrechner ausgeführt zu werden und dabei ein anderes Zielsystem zu debuggen. Das wird über das GDB RSP (Remote Serial Protocol) und einen sogenannten Debugging Stub realisiert. Der Stub läuft auf dem Zielsystem und bildet die Schnittstelle zwischen GDB und dem zu untersuchendem System. Dadurch kann GDB das Zielsystem steuern und dessen Zustand analysieren, ohne dass der Debugger vollständig implementiert werden muss. Das Zielsystem muss besondere Subroutinen implementieren um mit GDB zu kommunizieren und die empfangenen Befehle auszuführen [1]. In dieser Arbeit wird die Kommunikation über den seriellen Port und die Auswertung der Pakete durch die Rust Crate `gdbstub` umgesetzt. Die Crate implementiert das GDB RSP und bietet Schnittstellen an, in denen man das Verhalten des Zielsystems definieren muss [4].

2.5 Verwandte Projekte

Der Autor der Crate verweist in der Dokumentation auf eine Sammlung beispielhafter Implementierungen. Die Beispiele bieten einen ersten Überblick über die Crate für verschiedene Anwendungsfällen und Architekturen [4]. Außerdem gibt es eine Implementierung des GDB RSP ohne die Crate für hhuOS [10].

2.6 Umsetzung von GDB Befehlen über das Remote Serial Protocol

Nicht jedem GDB Befehl ist auch ein entsprechendes Paket zugeordnet. Manche Befehle werden durch mehrere verschiedene Pakete von GDB zusammengesetzt. Durch den Befehl `set debug remote 1` zeigt GDB die geschickten und empfangenen Pakete nach jedem Befehl an [1]. Im Folgenden wird die Funktionsweise der wichtigsten Befehle erläutert.

2.6.1 Lesen & Schreiben von Speicher

Um Variablen zu lesen und zu schreiben nutzt GDB beim remote Debugging eine Kombination aus den Register- und Adressfunktionen. Wenn man zum Beispiel in einer Funktion `foo` einen Breakpoint trifft und an GDB meldet, dann holt sich GDB zusätzlich noch den Registerzustand und liest einen Teil vom Stack aus. Wenn man dann eine lokale Variable `bar` auslesen möchte, berechnet GDB die Adresse als Kombination aus dem Wert vom Base Pointer und einem Offset. Danach kann GDB den Wert aus dem Stackbereich direkt auslesen, ohne weitere Pakete an den Stub zu schicken.

2.6.2 Continue von einem Breakpoint

Bei Software Breakpoints auf der x86-64 Architektur wird das erste Byte der Instruktion an der Stelle des Breakpoints durch die Breakpointinstruktion `0xCC` ersetzt. Wird diese

Instruktion ausgeführt, löst die CPU eine Breakpoint Exception aus. Dabei zeigt der Instruction Pointer auf die Adresse hinter der Breakpointinstruktion. Damit das Programm nach dem Stopp korrekt fortgesetzt werden kann, sind folgende Schritte nötig [11]:

1. Instruction Pointer um eins reduzieren, weil er auf die Instruktion nach dem Breakpoint zeigt
2. Breakpoint durch Originalbyte ersetzen
3. Trap Flag setzen, damit nach einer Instruktion wieder eine Exception ausgelöst wird
4. Instruktion ausführen
5. Breakpoint wieder zurückschreiben

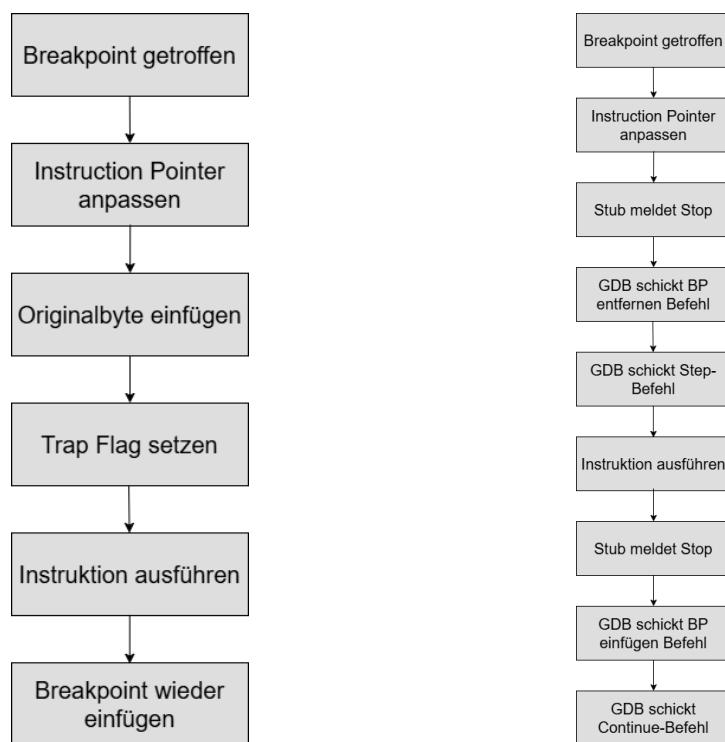
GDB übernimmt aber Teile der Logik, sodass nicht alles im Breakpoint-Handler gemacht werden muss. Wenn der Stub meldet, dass ein Breakpoint getroffen wurde, liest GDB Register und Speicher aus und entfernt dann den Breakpoint. Das Originalbyte wird also durch die `remove_sw_breakpoint` Funktion des Stubs zurückgeschrieben. Wenn man jetzt vom Breakpoint aus fortfahren will, schickt GDB typischerweise zuerst einen Step-Befehl, es soll also nur die nächste Instruktion ausgeführt und dann eine Debug Exception ausgelöst werden. Sobald GDB wieder die Kontrolle nach dem Step bekommt, setzt es den Breakpoint erneut ein und schickt dann entweder einen Continue-Befehl, wenn der Nutzer davor selber einen Continue-Befehl eingegeben hat oder wartet auf weitere Anweisungen. Der Breakpoint-Handler muss also noch den Instruction Pointer reduzieren, während der Rest der Logik durch verschiedene GDB Pakete zusammengesetzt und die entsprechenden Funktionen des Stubs ausgeführt wird. Die Unterschiede zwischen dem Handler im Stub und einem normalen Handler kann man auch in Grafik 2.1 sehen.

2.6.3 Step Into

Durch den Befehl `step` oder `s`, wird das Zielprogramm ausgeführt, bis es die nächste Zeile im Quellcode erreicht. Sollten dafür mehrere Instruktionen ausgeführt werden müssen, schickt GDB auch mehrere `step` Befehle und liest den Speicher aus, bis es erkennt, dass die nächste Zeile erreicht wurde. Dabei werden Funktionsaufrufe nicht direkt ausgeführt, sondern GDB schreitet in die Funktion. Das wird über temporäre Software Breakpoints realisiert. Wenn GDB den Funktionsaufruf sieht, setzt es einen Software Breakpoint an der Adresse der aufgerufenen Funktion und lässt das Programm weiter laufen, bis der Breakpoint erreicht wird.

2.6.4 Step Over

Mit dem Befehl `next`, beziehungsweise `n`, wird das Zielprogramm ebenfalls bis zur nächsten Zeile im Quellcode ausgeführt. Dabei werden Funktionen aber nicht betreten, sondern direkt vollständig ausgeführt. Dazu setzt GDB einen Software Breakpoint auf die Zeile nach dem Funktionsaufruf. Danach wird das Zielprogramm weiter ausgeführt bis es den Breakpoint erreicht. Es wird also alles bis zur nächsten Zeile in der aktuellen Funktion ausgeführt.



(a) Breakpoint Behandlung ohne Stub (b) Breakpoint Behandlung mit Stub

Abbildung 2.1: Unterschiede Breakpoint Behandlung

Kapitel 3

Implementierung

Wie eben bereits erwähnt, wird in dieser Arbeit die Crate `gdbstub` verwendet. Sie implementiert das GDB RSP, sodass nicht jedes Detail der Paketverarbeitung manuell umgesetzt werden muss. Stattdessen bietet die Crate Schnittstellen, über die das Zielsystem sein spezifisches Verhalten bereitstellt. Eine minimale Integration erfordert lediglich, dass man die Traits `Connection` und `Target`, sowie eine Form des Event Loops implementiert. In `Connection` wird die Kommunikationsart, hier der serielle Port, definiert. Der `Target` Trait dient als primäre Brücke zwischen dem RSP und dem Systemspezifischen Code. Hier wird definiert, wie das System während der Debugging Session kontrolliert und modifiziert werden kann. Der Event Loop verarbeitet die eingehenden Pakete und leitet sie an die entsprechenden Schnittstellen weiter, schickt aber auch Pakete an das RSP zurück bei Events, die durch zum Beispiel Systemstopp ausgelöst werden [4].

3.1 Connection

Der `Connection` Trait ist die Schnittstelle zur Kommunikation. Da D3OS eine No-Std Umgebung ist, muss `Connection` manuell implementiert werden. Dazu müssen die Methoden `write()` und `flush()` mit dem Verhalten unserer Kommunikationsart befüllt werden [4]. Es wurde sich hier für einen polling-basierten seriellen Port entschieden, da D3OS schon die Schnittstelle für den seriellen Port bereitstellt und die Rechner der Abteilung Betriebssysteme über einen solchen Port verfügen. Da es sich bei polling-basierenden Verfahren um blockierende I/O handelt, muss hier zusätzlich der Trait `ConnectionExt` implementiert werden, der neben den eben genannten Methoden weiterhin `read` und `peek` benötigt [4].

3.2 BlockingEventLoop

Der Event Loop ist einerseits dafür verantwortlich, eingehende Pakete auszuwerten und die richtigen Funktionen aufzurufen, andererseits aber auch dafür den Systemzustand an GDB zu übermitteln, wenn bestimmte Ereignisse passieren. Die Crate bietet dafür zwei Möglichkeiten an. Mit der `GdbStubStateMachine` hat man selbst die volle Kontrolle über den Event Loop. Für Implementierungen, die keinen extra Thread aufwenden wollen beziehungsweise können oder Interrupt getriebene I/O verwenden, ist das die bevorzugte Variante [4].

Da hier aber schon blockierende I/O verwendet wird und beim debuggen auch nicht die

maximale Effizienz des Systems benötigt wird, wird hier der `BlockingEventLoop` verwendet. Wie der Name schon sagt blockiert der Event Loop so lange, bis entweder neue Daten anliegen oder der Stub ein Event and GDB melden muss. Dazu muss die Methode `wait_for_stop_reason` implementiert werden. Außerdem erwartet der `BlockingEventLoop` Trait auch, dass die Methode `on_interrupt` implementiert wird [4].

3.2.1 `wait_for_stop_reason`

Diese Methode wird aufgerufen nachdem die `resume` Methode des Zielsystems aufgerufen wurde. Sie soll blockieren bis entweder neue Daten über `Connection` gesendet wurden oder das System gestoppt ist [4]. Dazu läuft eine Endlosschleife, in der zuerst mit `peek` überprüft wird, ob neue Daten anliegen und falls ja werden diese gelesen und über `Event::IncomingData(byte)` an die entsprechenden Methoden weitergeleitet. Danach wird überprüft, ob der `DebugState` ein `DebugEvent` gespeichert hat. Falls ja wird basierend auf dem `DebugEvent` entschieden, welche `StopReason` an GDB übermittelt wird.

3.2.2 `on_interrupt()`

Diese Funktion verarbeitet den `CtrlC` Befehl von GDB. Dieser Befehl soll den Programmfluss anhalten und GDB die Kontrolle geben [1]. Dazu werden die Interrupts ausgeschaltet und alle Threads außer dem Stub-Thread angehalten. Außerdem wird ein `DebugEvent::CtrlC` gesetzt. Danach wird zum Stub-Thread gewechselt und es können neue Befehle eingegeben werden. Das Programm wird hier aber nicht durch eine Exception unterbrochen. Der `CtrlC` Befehl wird erst verarbeitet, wenn der Scheduler zum Stub-Thread wechselt. Dieser hält das Programm dann an und wartet auf neue Befehle. Da erst zum Stub-Thread gewechselt werden muss, befindet sich der angehaltene Thread immer im Kontext der `thread_switch` Funktion.

3.3 `DebugState`

Für die Kommunikation zwischen den verschiedenen Komponenten und dem Stub wird ein globaler Zustand verwendet. Dieser ist notwendig, da Exceptions im Kontext des ausgeführten Threads auftreten und behandelt werden, die Kommunikation mit GDB jedoch durch den Stub-Thread erfolgt. Der globale Zustand speichert das zuletzt aufgetretene `DebugEvent`, damit es vom Event Loop verarbeitet werden kann, aber auch Listen der gesetzten Breakpoints. Da auf den Zustand aus verschiedenen Threads zugegriffen wird, werden die einzelnen Felder über Mutexe synchronisiert.

3.4 `Target`

Der `Target` Trait ist eine zentrale Schnittstelle der Crate. Dazu stellt `gdbstub` verschiedene Handler-Methoden bereit, die vom Zielsystem implementiert werden müssen. Hier wird unterschieden zwischen Single-Thread- und Multi-Thread-Systemen. Da D3OS mehrere Threads unterstützt, werden in dieser Arbeit die Schnittstellen für Multi-Thread-Zielsysteme implementiert. Eine minimale Implementierung erfordert hier die Methoden `read_registers`, `write_registers`, `read_addrs`, `write_addrs` und `list_active_threads`.

Das ist aber um diverse weitere Funktionen erweiterbar.

Außerdem müssen hier auch architekturspezifische Informationen wie Pointergröße und Registerlayout angegeben werden [4]. Für viele der gängigen Architekturen, unter anderem auch das von D3OS verwendete x86_64, gibt es aber eine zusätzliche Crate `gdbstub_arch`, in welcher die erforderlichen Informationen bereits implementiert sind [12].

3.4.1 Inlineable Dyn Extension Traits

Das GDB RSP ist sehr umfangreich und bietet viele verschiedene optionale Funktionen an. Damit nicht jeder Entwickler jedes optionale Feature implementieren muss, verwendet `gdbstub` optionale Trait Erweiterungen. Da die klassischen Ansätze optionale Methoden zu implementieren Nachteile hinsichtlich der Performance oder Ergonomie mit sich bringen, verwendet `gdbstub` eine Technik die der Autor *Inlineable Dyn Extension Traits* (IDETs) nennt. Dadurch lassen sich die Traits problemlos erweitern und der Compiler erkennt Dinge wie gegenseitig abhängige Methoden bereits zur Compilezeit, ohne dass Ummengen von Cargo Features benutzt werden müssen. Die genaue Funktionsweise der IDETs kann in der Dokumentation der Crate nachgelesen werden [4].

3.4.2 read_registers

Die Methode `read_registers` bekommt als Parameter eine Thread-ID und eine Struct, deren Felder mit dem Registerzustand des Threads befüllt werden müssen, übergeben [4]. Dazu werden zuerst die Threadinformationen durch den Scheduler abgerufen. Über die Thread Struct von D3OS kann der zuvor gespeicherte Stack Pointer `old_rsp0` des Threads abgefragt werden. Da beim Verarbeiten von GDB Anfragen der Thread mit dem Stub aktiv ist, liegt der gesamte Threadkontext wie in Grafik 3.1 auf dem Stack. Dabei zeigt `old_rsp0` auf den gespeicherten Wert von `GSBASE`, also genau den Anfang des Bereichs, in dem der Registerzustand gespeichert ist. Die Registerwerte liegen als 64 Bit Integer auf dem Stack. Daher kann `old_rsp0` als Pointer auf ein Array interpretiert werden. Im Falle von `read_registers` wird von diesem Array eine Kopie erstellt, da hier nur gelesen werden muss. Außerdem erwartet GDB noch den Stack Pointer, der nach Wiederherstellung des Threadzustands gültig ist. Dieser kann berechnet werden, indem `old_rsp0` um die Größe eines 64 Bit Integers, multipliziert mit der Anzahl der auf dem Stack liegenden Werte, erhöht wird. Mit dem Array und dem berechneten Stack Pointer Wert kann anschließend die übergebene Struct befüllt werden, über die der Registerzustand an GDB gemeldet werden kann.

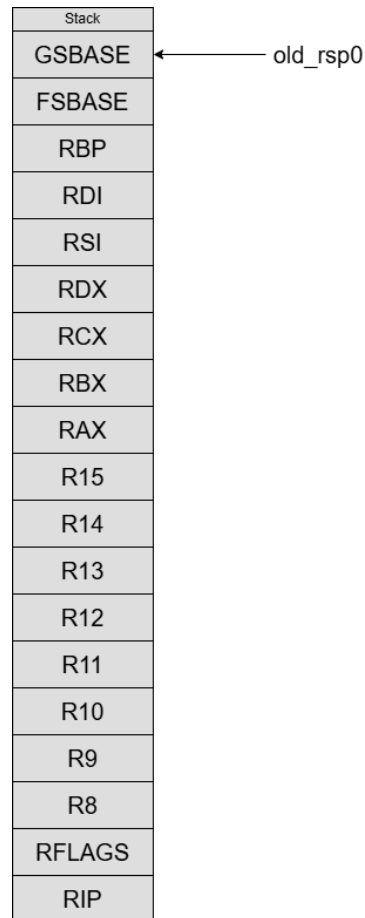


Abbildung 3.1: Registerwerte auf dem Stack

```

gdbtarget.rs Rust
1 pub fn thread_context_from_rsp(rsp: VirtAddr) -> Option<ThreadContext> {
2     if rsp.is_null() {
3         return None;
4     }
5     let raw_rsp0 = rsp.as_u64();
6     let registers = unsafe {
7         (raw_rsp0 as *const [u64; THREAD_REG_COUNT]).read()
8     };
9     let restored_rsp = raw_rsp0 + (THREAD_REG_COUNT * size_of<u64>()) as u64;
10    Some(ThreadContext {
11        registers,
12        rsp: restored_rsp,
13    })
14 }

```

3.4.3 write_registers

Die Methode `write_registers` ist ähnlich zu `read_registers` aufgebaut. GDB übergibt hier wieder eine Thread-ID und eine Struct mit Registerwerten. Im Gegensatz zu `read_registers` muss jedoch der Threadzustand mit den Werten aus der Struct überschrieben werden [4]. Dazu werden wieder die Threadinformationen durch den Scheduler abgerufen und über `old_rsp0` der Speicherbereich auf dem Stack als Array interpretiert.

Für `write_registers` wird aber keine Kopie erstellt, sondern eine mutable Referenz auf den Speicherbereich. Dadurch können die Werte auf dem Stack direkt verändert werden. Diese Werte werden beim nächsten Kontextwechsel in die Register geladen.

```
gdbtarget.rs Rust
1 pub fn mut_thread_context_from_rsp(rsp: VirtAddr) -> Option<ThreadContextMut> {
2     // Das meiste ist gleich wie bei der Read-Only Version,
3     // der Hauptunterschied ist in folgender Zeile
4     let registers = unsafe {
5         &mut *(raw_rsp0 as *mut [u64; THREAD_REG_COUNT])
6     };
7 }
```

3.4.4 read_addr

GDB übergibt hier eine virtuelle Adresse, eine Thread-ID und ein Array `data`, das mit den Daten an der jeweiligen Adresse befüllt werden muss [4]. Da der Stub vorerst nur im Kernel arbeitet, muss hier kein neuer Adressraum geladen werden. Man kann also direkt über die Bytes, die an der virtuellen Adresse stehen, iterieren und die Daten in das Array schreiben. Die Funktion meldet bei erfolgreicher Ausführung `data.len()` an GDB.

3.4.5 write_addr

Die Methode `write_addr` funktioniert ähnlich zu `read_addr`, das `data` Array hält hier aber die Daten, die an die Adresse geschrieben werden sollen [4]. Die Daten an der Adresse werden also beim iterieren überschrieben statt gelesen.

3.4.6 list_active_threads

GDB übergibt hier eine Callback Funktion, an die die IDs aller aktiven Threads übergeben werden müssen [4]. Der Scheduler verwaltet die aktiven Threads in Listen, die nach Zustand sortiert sind. Durch diese Listen kann iteriert werden, um die IDs der Threads, die nicht im Zustand `Exited` sind, abzurufen. Die IDs werden an die Callback Funktion übergeben und dadurch an GDB weitergeleitet.

3.5 Software Breakpoints

Breakpoints sind eine zentrale Funktion eines Debuggers um Programme richtig untersuchen zu können. Software Breakpoints können durch Einsetzen einer Architekturspezifischen Instruktion verwendet werden und lösen eine Exception aus, wenn diese Instruktion ausgeführt wird [11]. `gdbstub` bietet jeweils eine Funktion zum setzen und entfernen von Software Breakpoints. Das genaue Verhalten des Systems, nachdem ein Breakpoint getroffen wurde, muss jedoch von einer extra Handler Funktion implementiert werden.

3.5.1 add_sw_breakpoint

Der Stub bekommt hier von GDB eine virtuelle Adresse und ein `BreakpointKind` übergeben [4]. `BreakpointKind` wird normalerweise benutzt, um auf bestimmten Architekturen

zu entscheiden, welche Art Breakpoint eingesetzt werden soll [1]. Bei `x86_64` gibt es aber nur eine Software Breakpoint Instruktion, also ist dieser Parameter hier egal [11]. Um Software Breakpoints zu verwalten, wird eine Liste im `DebugState` gespeichert, die die Adressen und Originalbytes für die Breakpoints sichert. Da es kein Limit für gleichzeitig aktive Software Breakpoints gibt, wird als Liste hier ein Vector verwendet, damit sie dynamisch wachsen kann. Zuerst wird überprüft, ob in der Liste von Breakpoints bereits ein Breakpoint an der übergebenen Adresse steht. Falls ja, wird GDB direkt informiert, dass dort ein Breakpoint steht. Das ist die gleiche Antwort, die auch beim erfolgreichen Einsetzen eines neuen Breakpoints geschickt wird. Wenn die Adresse noch nicht in der Liste ist, wird erst das erste Byte der Originalinstruktion an der Stelle ausgelesen. Danach wird an diese Stelle die Breakpointinstruktion `0xCC` geschrieben [11]. Dann muss das Byte der Originalinstruktion mit der Adresse zusammen in die Liste mit Breakpoints geschrieben werden, damit später die richtige Instruktion wieder an die Adresse geschrieben werden kann.

3.5.2 `remove_sw_breakpoint`

Die Methode bekommt die gleichen Parameter wie `add_sw_breakpoint` [4]. Hier wird in der Liste nach dem Eintrag mit der richtigen Adresse gesucht. Der Eintrag wird gelöscht und die Originalinstruktion wieder an die Adresse geschrieben.

3.5.3 Breakpoint Handler

Wenn eine Breakpoint Instruktion ausgeführt wird, löst das System einen INT3 Interrupt, beziehungsweise eine Breakpoint Exception, aus [11]. D3OS verwendet zum behandeln von Exceptions den IDT (Interrupt Descriptor Table) aus der `x86_64` Crate [13]. Dadurch können über ein Makro sehr einfach Handler-Funktionen für bestimmte Exceptions registriert werden. Das Makro bekommt die IDT Instanz, die Handler-Funktion und den Index des Interrupts, der behandelt werden soll, übergeben. Der Index ist hier drei. Die Handler-Funktion bekommt dann ein `InterruptStackFrame` übergeben, welches die folgende Werte enthält [13]:

- Instruction Pointer
- Code Segment
- RFlags
- Stack Pointer
- Stack Segment

Damit die Threads nicht weiterlaufen, müssen beim Betreten des Breakpoint-Handlers Interrupts ausgeschaltet werden. Das reicht aber nicht, da die Threads nach Verlassen des Handlers trotzdem weiter laufen. Sie werden aber nicht mehr richtig durch den Scheduler verwaltet, was dazu führt, dass die Threads fälschlicherweise terminieren. Um das zu vermeiden wird hier ein neuer Thread Zustand `DebugStopped` hinzugefügt. Im Breakpoint-Handler werden alle Threads, die als Zustand `Ready` haben, zu `DebugStopped` geändert und der `blocked_list` hinzugefügt, damit sie nicht vom Scheduler ausgewählt werden

können sobald der Handler verlassen wird. Der Thread, in dem der Stub läuft, darf aber nicht gestoppt werden. Außerdem muss der Stub an GDB melden, dass ein Breakpoint getroffen wurde. Dazu wird ein `DebugEvent::SwBreakpoint` in den `DebugState` geschrieben. Da der Breakpoint-Handler durch den Thread, der die Exception ausgelöst hat, aufgerufen wird, muss zum Schluss noch zum Stub-Thread gewechselt werden. Er ist der einzige Thread mit Zustand `Ready`, weshalb hier einfach `switch_thread_from_interrupt()` aufgerufen werden kann, weil es nur einen Thread gibt, in den der Scheduler wechseln kann. Der Thread mit dem Stub läuft dann in einem Busy Loop im Exceptionkontext, bis neue Befehle von GDB ankommen. Außerdem muss der Handler den Instruction Pointer noch reduzieren.

Der `InterruptStackFrame`, den der Handler übergeben bekommt, ist nur ein Wrapper für die Werte, damit nicht versehentlich etwas geändert wird. Er kann jedoch in ein `InterruptStackFrameValue` konvertiert werden, in dem sich die Werte dann ändern lassen. Beim Testen hat sich jedoch rausgestellt, dass das nur Kopien der Werte sind und Änderungen nur die Instanz betreffen. Deswegen werden Änderungen von Flags oder dem Instruction Pointer nicht übernommen.

Weil beim Interrupt andere Werte gepusht werden, als bei einem Thread Switch, kann die Funktion, die den Threadkontext vom alten Stack Pointer beginnend ausliest, nicht verwendet werden. Die Werte, die dort auf dem Stack liegen, sind nicht die Register des Threads. Wenn man zum Beispiel versucht eine Trap Flag an die Stelle zu schreiben, an der sonst das RFlags Register wäre, wird entweder eine Page Fault oder eine General Protection Fault ausgelöst, was zum Absturz von D3OS führt.

Das kann man umgehen, indem man den Handler nicht über das Makro registriert, sondern direkt über ein Feld in der IDT Instanz. Die Funktion bekommt dadurch auch eine Instanz der gleichen `InterruptStackFrame` Struct, hier sind die Werte aber tatsächlich veränderbar und nicht bloß Kopien der Werte.

```
interrupt_dispatcher.rs
```

```
Rust
```

```
1 let mut idt = idt().lock();
2
3 //Makro
4 set_general_handler!(&mut idt, gdb_handle_int3, 3);
5
6 //Direkt über Feld der IDT Instanz
7 idt.breakpoint.set_handler_fn(gdb_handle_int3);
```

Jetzt lassen sich zwar die Werte aus dem `InterruptStackFrame` ändern und das Programm verhält sich nach dem Breakpoint weiter wie erwartet, GDB liest aber den Speicher nicht richtig aus. Wie eben bereits erwähnt, liest GDB Speicher aus, indem es erst die Register ausliest und dann einen Teil vom Stack ausgehend vom Base Pointer. Da `read_registers` aber über den gespeicherten Stack Pointer die Werte ausliest, funktioniert das in einem Exceptionkontext nicht. Das ist das gleiche Problem wie bei den Flags und dem Instruction Pointer. Der Stack von `old_rsp0` aus beinhaltet hier nicht mehr die Werte, die erwartet werden. Damit der Threadkontext richtig ausgelesen werden kann, müssen die Registerwerte zusätzlich zum `InterruptStackFrame` auf den Stack geschrieben werden. Dazu muss der Handler wieder anders registriert werden. Statt `set_handler_fn` kann man `set_handler_addr` verwenden [13]. Dadurch kann eine Assembler Funktion statt einer Rust Funktion registriert werden. Der IDT pusht bereits die Werte für den `InterruptStackFrame`, also müssen noch die anderen Register

durch die Assembler Funktion gepusht werden. Außerdem wird hier eine weitere Rust Funktion `gdb_interrupt_handler` aufgerufen, die als Argumente den gepushten Kontext als `GdbTrapFrame` und den Exception Index bekommt. Hier wird anhand des Index entschieden, welcher spezifische Handler aufgerufen wird und ihm wird als Argument eine mutierbare Referenz auf den `GdbTrapFrame` übergeben. In diesem Handler wird dann der Breakpoint behandelt und der Trap Frame in dem Thread gespeichert, der die Exception ausgelöst hat. Die Funktionen zum Register lesen und schreiben werden dann so erweitert, dass sie zuerst überprüfen, ob der Thread ein Trap Frame gespeichert hat und falls ja, wird der Kontext nicht über den `old_rsp0` erstellt, sondern das Trap Frame verwendet. Dadurch kann auch im Exceptionkontext der Speicher richtig ausgelesen werden.

interrupt_dispatcher.rs

Rust

```

1 let mut idt = idt().lock();
2
3 //Handler Fn
4 idt.breakpoint.set_handler_fn(gdb_handle_int3);
5
6 //Handler Addr
7 unsafe {
8     idt.breakpoint.set_handler_addr(VirtAddr::new(gdb_breakpoint_entry as
9         u64));
10 }

```

gdbtarget.rs

Rust

```

1 #[unsafe(no_mangle)]
2 pub extern "C" fn gdb_interrupt_handler(frame: *mut GdbTrapFrame, index: u64) {
3     let frame = unsafe { &mut *frame };
4
5     match index {
6         1 => gdb_handle_debug_exception(frame),
7         3 => gdb_handle_int3(frame),
8         _ => panic!("unexpected gdb trap index {}", index),
9     }
10 }

```

gdb_interrupt_handler.asm

Assembler

```
global gdb_breakpoint_entry
extern gdb_interrupt_handler
```

```
gdb_breakpoint_entry:
```

```
push r15
push r14
push r13
push r12
push r11
push r10
push r9
push r8
push rsi
push rdi
push rbp
push rdx
push rcx
push rbx
push rax
```

```
mov rdi, rsp
mov rsi, 3
mov rbx, rsp
and rsp, -16
call gdb_interrupt_handler
mov rsp, rbx
```

```
pop rax
pop rbx
pop rcx
pop rdx
pop rbp
pop rdi
pop rsi
pop r8
pop r9
pop r10
pop r11
pop r12
pop r13
pop r14
pop r15
```

```
iretq
```

3.6 MultiThreadResume

Um von einem Programmstop aus wieder fortfahren zu können, kann man entweder `Continue` oder `Step` verwenden. Da D3OS mehrere Threads unterstützt, können auch verschiedene Befehle für verschiedenen Threads angegeben werden. Um zu unterscheiden, wie welcher Thread fortgeführt werden soll, verwendet `gdbstub` eine Liste von `ResumeActions`. Diese Liste wird in der `GdbStubTarget` Struct gespeichert. Es kann jedoch auch sein, dass GDB nicht explizit eine `ResumeAction` für einen Thread angibt, dann ist das Standardverhalten den Thread wie bei einem `Continue` laufen zu lassen [4].

3.6.1 clear_resume_actions

Damit nicht versehentlich alte Befehle für die Threads verwendet werden, ruft GDB zuerst `clear_resume_actions` auf, gefolgt von null oder mehr Aufrufen von den verschiedenen `set_resume_action` Funktionen [4]. Also wird in dieser Funktion die Liste einfach geleert, unabhängig davon, was drin steht.

3.6.2 set_resume_action_continue

Die Funktion bekommt eine Thread-ID übergeben, die dann zusammen mit der `ResumeAction::Continue` in der Liste gespeichert wird. Wenn man den `MultiThreadResume` Trait implementiert, muss mindestens `Continue` implementiert werden [4].

3.6.3 set_resume_action_step

Diese Funktion ist identisch zu `set_resume_action_continue`, nur dass hier `ResumeAction::SingleStep` gesetzt wird [4]. Stepping ist zwar eine optionale Funktion für `MultiThreadResume`, sollte aber trotzdem mit implementiert werden. Wie eben bereits erklärt, schickt GDB einen Step-Befehl an den Stub, auch wenn man eigentlich einen Continue-Befehl geschickt hat, um einen Breakpoint zu behandeln. Um Breakpoints richtig benutzen zu können braucht man also sowohl `Continue`, als auch `Step`.

3.6.4 set_resume_action_scheduler_lock

Wenn diese Funktion aufgerufen wird, soll das Resume-Verhalten verändert werden. Statt alle Threads wie bei einem `Continue` fortzuführen, wenn keine explizite `ResumeAction` gesetzt wurde, dürfen nur Threads wieder gestartet werden, wenn auch tatsächlich eine `ResumeAction` vorliegt [4]. Dazu wird ein Boolean im `DebugState` gespeichert, der beim Aufruf der Scheduler-Lock Funktion umgeschaltet wird. Basierend darauf wird am Ende der Resume-Funktion entschieden, ob alle Threads fortfahren dürfen oder nicht. Scheduler-Locking ist zwar auch laut Crate optional aber wenn man versucht den Stub ohne den Trait laufen zu lassen stürzt das System ab, weil es erwartet, dass Scheduler-Locking implementiert ist.

3.6.5 Resume

Nachdem GDB die `ResumeActions` gesetzt hat, wird `resume` aufgerufen. Die Threads werden dann entsprechend ihrer `ResumeAction` behandelt. Es kann sein, dass die Liste

der Actions leer ist. Dann werden alle Threads wie bei einem **Continue** fortgeführt. Sollte mindestens ein Eintrag in der Liste sein, wird die Liste durchlaufen und für jeden Thread die spezifische Action angewendet.

Continue

Hier werden die Threads durch den Scheduler von **DebugStopped** wieder zu **Ready** geändert. Dadurch können sie nach dem Verlassen der Funktion wieder normal weiter laufen, bis ein neuer Stoppgrund durch den Stub gemeldet wird, zum Beispiel ein Software Breakpoint.

SingleStep

Bei einem **SingleStep** muss für den entsprechenden Thread die Trap Flag gesetzt werden. Diese CPU Flag sorgt dafür, dass nach der nächsten Instruktion eine Debug Exception ausgelöst wird. Um sie zu aktivieren muss man Bit 8, oder den Wert 0x100, im RFlags Register des jeweiligen Threads setzen [11]. Sollte ein **GdbTrapFrame** für den Thread existieren, zum Beispiel durch einen Breakpoint der getroffen wurde, wird die Flag über diesen Kontext gesetzt. Sollte jedoch kein Trap Frame gespeichert sein, wird der Kontext über den `old_rsp0` erstellt und die Trap Flag dort eingesetzt. Danach wird der Thread durch den Scheduler wieder auf **Ready** gesetzt.

3.6.6 Debug Exception Handler

Nachdem eine Instruktion mit gesetzter Trap Flag ausgeführt wurde, wird eine Debug Exception ausgelöst [11]. Damit das Programm nach der Exception nicht weiter läuft und GDB wieder die Kontrolle bekommt, muss sie entsprechend behandelt werden. Dazu wird ein weiterer Exception Handler registriert, so wie auch schon der Breakpoint Handler. Der Index für eine Debug Exception ist eins [11]. Wie auch beim Breakpoint Handler, wird eine Assembler Funktion gebraucht, die alle Register auf den Stack legt. Nach einem Single Step soll GDB wieder die Kontrolle bekommen, also werden wie bei Breakpoints auch Interrupts deaktiviert und alle Threads außer dem Stub-Thread auf **DebugStopped** gesetzt. Die Trap Flag wird entfernt und ein `DebugEvent::SingleStep` gesetzt. Danach wird zum Stub-Thread gewechselt.

3.7 Hardware Breakpoints

Hardware Breakpoints sind, anders als Software Breakpoints, auf vier Breakpoints gleichzeitig limitiert. Um Hardware Breakpoints zu setzen, werden die Debug Register verwendet. Die Debug Address Register DR0 bis DR3 speichern die virtuellen Adressen von Hardware Breakpoints. Wenn die Adresse in einem der Register steht, muss der Breakpoint zusätzlich noch im Debug Control Register DR7 aktiviert werden. Hier können auch verschiedene Bedingungen für die Breakpoints eingestellt werden, zum Beispiel kann auf Speicherzugriffen an der Adresse eine Exception ausgelöst werden, statt nur auf Instruktionsausführung [11]. Das ist aber nur für Hardware Watchpoints relevant, da für GDB Hardware Breakpoints, wie Software Breakpoints, nur bei Instruktionsausführung auslösen sollen [1]. Die Breakpoints können entweder Lokal, also nur für den aktuellen Thread,

oder Global für alle Threads aktiviert werden [11]. Das GDB RSP schickt aber keine Thread ID mit beim setzen von Breakpoints [1], also werden im Fall von diesem Stub Hardware Breakpoints Global aktiviert. Wenn ein Breakpoint getroffen wird, wird eine Debug Exception generiert [11]. Der Debug Exception Handler, der auch Single Stepping behandelt, muss also erweitert werden, um zu erkennen, ob die Exception durch einen Breakpoint oder eine Trap Flag ausgelöst wurde. Dazu wird das Debug Status Register DR6 verwendet, was unter anderem speichert, welches der Debug Address Register die Debug Exception ausgelöst hat. Da Hardware Breakpoints über Register, statt über eine Instruktion, gesetzt werden, können sie auch in Adressräumen gesetzt werden, die nicht geladen sind. Es können also Breakpoints im User-Space gesetzt werden, obwohl Software Breakpoints aktuell nur im Kernel-Space funktionieren. Von einem Breakpoint im User-Space kann aber kein Single Step ausgeführt werden, da dafür der Adressraum wieder geladen sein muss.

3.7.1 `add_hw_breakpoint`

GDB übergibt dem Stub eine virtuelle Adresse, wie auch bei Software Breakpoints [4]. Die gesetzten Hardware Breakpoints werden auch in einer Liste gespeichert im `DebugState`. Da Hardware Breakpoints aber auf maximal vier limitiert sind, reicht hier ein Array mit vier Plätzen, statt einem Vector wie bei den Software Breakpoints. Zuerst wird überprüft, ob die Adresse bereits in dem Array gespeichert ist, damit nicht zwei mal der gleiche Breakpoint gesetzt wird. Zusätzlich muss überprüft werden, ob überhaupt noch ein Hardware Breakpoint gesetzt werden kann, oder ob bereits 4 Breakpoints aktiviert sind. Dazu wird auch das Array durchlaufen, um einen freien Platz zu finden. Wenn ein freier Platz im Array gefunden wurde, wird mit über den Index entschieden, welches Debug Address Register für den Breakpoint verwendet wird. Die Adresse wird in das entsprechende Register geschrieben und der Breakpoint im Debug Control Register aktiviert.

3.7.2 `remove_hw_breakpoint`

Um einen Hardware Breakpoint wieder zu entfernen, übergibt GDB wieder die virtuelle Adresse des Breakpoints [4]. Die Adresse wird in dem Array gesucht und durch 0 ersetzt. Über den Index im Array wird dann das entsprechende Register durch das Debug Control Register wieder deaktiviert. Das verwendete Debug Address Register muss nicht überschrieben werden, da der Breakpoint nicht ausgelöst werden kann, wenn er nicht in DR7 aktiviert ist. Die Adresse wird beim setzen von einem neuen Hardware Breakpoint ohne Probleme überschrieben.

3.7.3 Debug Exception Handler Erweiterung

Der Debug Exception Handler behandelt jetzt nicht nur Single Steps, sondern auch Hardware Breakpoints, da beides die gleiche Exception auslöst. Dazu muss im Handler unterschieden werden, wie die Exception ausgelöst wurde. Dazu kann das Debug Status register ausgelesen werden. Wenn ein Hardware Breakpoint getroffen wurde, ist ein entsprechendes Bit, je nachdem welches Debug Address Register den Breakpoint enthält, gesetzt [11]. Sollte eines dieser Bits gesetzt sein, wurde die Debug Exception durch einen Breakpoint, statt einer Trap Flag, ausgelöst. In dem Fall muss ein `DebugEvent::HwBreakpoint` zusammen

mit der Thread-ID im `DebugState` gesetzt werden und kein `DebugEvent::SingleStep`. Der Rest der Logik im Handler kann so bleiben wie vorher.

Kapitel 4

Benutzung

In diesem Kapitel wird die Verwendung von GDB zusammen mit dem Stub erklärt. Um den Stub zu verwenden, muss D3OS entweder in QEMU oder auf echter Hardware laufen. Wenn D3OS in QEMU ausgeführt wird, reicht ein zweites Terminal um GDB und den Stub zu verbinden. Dabei müssen in der QEMU konfiguration mindestens drei serielle Ports aktiviert sein, da auf `Com1` und `Com2` verschiedene Logger laufen. Außerdem muss der Stub auf `Com3` eingestellt sein.

Um den Stub auf echter Hardware verwenden zu können, müssen alle Logger und andere Komponenten, die den seriellen Port verwenden, abgeschaltet oder auskommentiert sein. Genauer dazu wird im Kapitel Evaluation beschrieben. Außerdem muss im BIOS des Rechners der richtige serielle Port ausgewählt werden, den der Stub verwendet. Außerdem müssen die beiden Rechner über ein entsprechendes Kabel verbunden sein. Der verwendete `ComPort` kann in der Datei `gdbconnection.rs` geändert werden.

4.1 Kompilieren

Der Stub wartet bei der Initialisierung darauf, dass GDB sich verbindet und hält dabei das System an. Damit man D3OS ohne eine Verbindung zu GDB verwenden kann, darf der Stub also nicht initialisiert sein. Um nicht immer händisch die Initialisierung aus dem Bootprozess entfernen zu müssen, ist sie durch **Cargo Features** abgesichert. Das bedeutet, dass der Aufruf der Initialisierungsfunktion und der Stub selbst nur kompiliert werden, wenn eine entsprechende Feature-Flag gesetzt ist [7]. Dazu kann beim kompilieren von D3OS der Befehl `cargo make -no-workspace gdbstub` verwendet werden.

4.2 Verwendung im Terminal

Wenn GDB im Terminal verwendet wird, kann eine Verbindung zum Stub über den Befehl `target remote [Gerät Name]` hergestellt werden [1]. Im Fall von QEMU ist der Befehl `target remote localhost:4321`, während er beim Testen auf Hardware `target remote \\dev\\ttyUSB0` war. Es kann außerdem sein, dass die Geschwindigkeit, mit der GDB Daten überträgt angepasst werden muss, wenn man den Stub auf Hardware verwendet. Dazu kann der Befehl `set serial baud [BaudRate]` verwendet werden [1]. Die BaudRate, auf die der Stub eingestellt ist, ist aktuell 115200. Zusätzlich muss eine Datei mit der entsprechenden Symboltabelle geladen werden. Für den Kernel geht das über den Befehl `file`

pfad/zur/datei/kernel.elf und für User-Space Anwendungen ,mit `add-symbol-file` pfad/zur/datei [1]. Das System bleibt angehalten, bis ein `Continue`-Befehl geschickt wird. Die verschiedenen Funktionen, die der Stub anbietet und die jeweiligen Befehle sind [1]:

- Verbindung aufbauen: `target remote [Gerät Name]`
- Debug Pakete: `set debug remote 1`
- ACK-Mode: `set remote noack-packet off`
- Übertragungsrate anpassen: `set serial baud [x]`
- Ausführung fortfahren: `continue` oder `c`
- Single Step (Step Into): `step` oder `s`
- Single Step (Step Over): `next` oder `n`
- Single Step (Maschineninstruktionen): `stepi/si` oder `nexti/ni`
- Software Breakpoint setzen: `break` gefolgt von der Adresse/Funktion/Zeile
- Hardware Breakpoint setzen: `hbreak` gefolgt von der Adresse/Funktion/Zeile
- Breakpoint entfernen: `delete [n]`
- Register lesen: `info registers`
- Variabelwert lesen: `print [Name]` oder `p [Name]`
- Speicher auslesen: `x/[n|Format|Größe] [Adresse]`, die ersten zwei Integer in einem Vec zum Beispiel: `x/2dw [Adresse]`
- Instanz-Felder auslesen: `p [InstanzName].[FeldName]`
- Ausführung anhalten: `Ctrl + C` drücken
- Funktion bei Step Into ignorieren: `skip` gefolgt von einer Datei/Funktion/Ausdruck

4.3 Verwendung mit VS Code

Der Stub kann auch mit der graphischen Benutzeroberfläche von Visual Studio Code verwendet werden. Dazu müssen die entsprechenden Extensions zur Nutzung von GDB installiert sein und eine Konfiguration für den Stub in der Datei `launch.json` angelegt werden. Abhängig davon, wie man sich verbinden will, muss `localhost:4321` durch das entsprechende Gerät ausgetauscht werden. Weitere Befehle, wie die Debug Ausgaben, können ebenfalls hinzugefügt werden, indem man `setupCommands` ergänzt.

launch.json

JSON

```
{ "name": "gdbstub",  
  "type": "cppdbg",  
  "request": "launch",  
  "program": "${workspaceFolder}/loader/kernel.elf",  
  "cwd": "${workspaceFolder}",  
  "miDebuggerPath": "rust-gdb",  
  "stopAtEntry": true,  
  "setupCommands": [  
    { "text": "-enable-pretty-printing" },  
    { "text": "set disassembly-flavor intel" },  
    { "text": "target remote localhost:4321" }  
  ]  
}
```

Kapitel 5

Evaluation

Ziel der Evaluation ist es zu prüfen, ob der implementierte GDB-Stub eine grundlegende Debugging-Sitzung mit D3OS ermöglicht. Dazu werden die implementierten Befehle des Stubs mithilfe von Testfunktionen, die in separaten Kernel-Threads aufgerufen werden, getestet. In den Testfunktionen werden lokale Variablen, weitere Funktionsaufrufe, Schleifen und Objektinstanzen verwendet. Die Tests wurden sowohl in einer emulierten Umgebung, als auch auf echter Hardware durchgeführt. D3OS wurde mit Debuginformationen kompiliert. Dazu wurde der Befehl `cargo make -no-workspace gdbstub` verwendet.

5.1 Breakpoints

Die Tests haben gezeigt, dass Software Breakpoints gesetzt, getroffen und entfernt werden können. Aktuell funktioniert das jedoch nur im Kernel-Space und nur auf Funktionen, die nicht in den Exception-Handlern aufgerufen werden, da der Breakpoint dort sonst erneut getroffen wird, bevor er entfernt werden kann, was zu einer unendlichen Schleife führt. Um Software Breakpoints auch im User-Space verwenden zu können, muss der Adressraum des jeweiligen Prozesses geladen werden. Hardware Breakpoints lassen sich jedoch auch im User-Space setzen. Außerdem kann es vorkommen, dass GDB mehrere Software Breakpoints auf die gleiche Zeile setzt. Bei Makro Aufrufen kann es sein, dass eine Zeile im Quellcode mehreren Adressen zugeordnet ist. Dann setzt GDB auf jeder der Adressen einen Breakpoint und trifft diese nacheinander.

5.2 Single-Stepping

Single-Stepping wurde durch die Befehle `step` und `next` nach einem getroffenen Breakpoint getestet. Dabei zeigte sich, dass Single-Stepping größtenteils so wie erwartet funktioniert. Man kann jedoch kein Step Into in Funktionen durchführen, die durch die Exception Handler des Stubs aufgerufen werden. Da GDB das Step Into durch temporäre Software Breakpoints realisiert, wird bei solchen Funktionen, wie zum Beispiel dem statischen Scheduler Aufruf, eine unendliche Exceptionschleife ausgelöst. GDB will in die Funktion eintreten und setzt einen Breakpoint auf die Funktion. Das Programm trifft den Breakpoint und betritt den Handler. Der Handler ruft aber ebenfalls die Funktion auf, während der Breakpoint noch aktiv ist, wodurch er wieder getroffen wird und der Handler neu betreten wird. Um nicht aktiv darauf achten zu müssen bestimmte Funktionen zu über-

springen, bietet GDB den `skip` Befehl an [1]. Dadurch lassen sich bestimmten Funktionen und auch ganze Dateien von einem Step Into ausschließen. Dazu muss der vollständige Pfad angegeben werden. Der Scheduler Aufruf aus der `lib.rs` Datei lässt sich zum Beispiel durch `skip -rfu kernel::scheduler` überspringen. Bei Makros ist das ganze jedoch schwieriger, weil nicht der Name des Makros verwendet werden kann, sondern die Funktionen des expandierten Makrocodes verwendet werden müssen. Das `info!()` Makro lässt sich zum Beispiel durch den Befehl `skip -rfu .*log::.*` überspringen. Dadurch überspringt GDB alle Funktionen, deren Name `log` beinhaltet.

Single-Stepping von einem Hardware Breakpoint im User-Space funktioniert jedoch nicht, da dafür wieder der Adressraum geladen sein muss.

5.3 Registerzugriff

Da viele der anderen Funktionen auf den Registerwerten basieren, müssen diese nicht isoliert getestet werden. Solange Single-Stepping und Speicherzugriffe korrekt funktionieren, werden auch die Register korrekt ausgelesen.

5.4 Speicher auslesen

Sowohl lokale Variablen, als auch Instanzen von Structs und deren Felder lassen sich auslesen. Die Zugriffe liefern jedoch nur das erwartete Ergebnis, wenn auch die Registerzustände korrekt ausgelesen werden und der Adressraum geladen ist.

5.5 Probleme des Serial Ports

D3OS verwendet für die Implementierung des Loggers ebenfalls den seriellen Port. Da die Bildungsrechner, auf denen getestet wurde, nur einen solchen Anschluss haben, führt das dazu, dass der Stub und die Logger sich Ressourcen teilen müssen. Damit der Stub und GDB sich zuverlässig austauschen können, müssen alle Logger ausgeschaltet sein und die Zuweisung des `ComPorts` im Konstruktor des Loggers muss ebenfalls auskommentiert sein. Wird das nicht gemacht, kann es zu unerwartetem Verhalten kommen. Dabei kann es sein, dass der Stub entweder falsche Pakete an GDB schickt und dadurch ein Error ausgelöst wird oder die Pakete, die von GDB verschickt werden, fehlerhaft ankommen, weil eine andere Komponente mit Zugriff auf den seriellen Port die Bytes konsumiert. Dieses Problem tritt in der emulierten Umgebung mit QEMU nicht auf, da dort in der Konfigurationsdatei mehrere serielle Anschlüsse eingestellt werden können.

Auf der neusten Version von D3OS reicht das aber auch nicht aus. Dort werden scheinbar trotzdem fehlerhafte Pakete ausgetauscht, obwohl alle Instanzen, die einen seriellen Port nutzen, ausgeschaltet oder auskommentiert sind. Durch die Debug-Ausgaben von GDB kann man beobachten, dass der Stub standardmäßig im `noack Mode` läuft. Das bedeutet, dass nach dem initialen Verbindungsaufbau keine **ACK-Pakete** mehr ausgetauscht werden. Wenn man diesen auf der neusten Version aber ausschaltet, kann man sehen, dass statt der **ACK-Pakete** teilweise Bytes oder auch ganze Pakete vom Stub an GDB zurückgeschickt werden. Der serielle Port verarbeitet also fehlerhaft die Pakete, was entweder zu einem `UnexpectedHeader` Fehler beim Verbindungsaufbau oder einem `MismatchedChecksum`

Fehler während der Benutzung führt. Beim testen konnte nicht rausgefunden werden, was genau dieses Verhalten auslöst.

```
[remote] Sending packet: $z0,1b8758b,1#04
[remote] Junk: z0,1b8758b,1
[remote] Junk: z0,1b8758b,1
[remote] Sending packet: $z0,1b8758b,1#04
[remote] Sending packet: $z0,1b8758b,1#04
```

Abbildung 5.1: Stub schickt Pakete zurück

```
[remote] Packet received: mp01.02,p01.03,p01.04,p01.05
[remote] Sending packet: $qsThreadInfo#c8
[remote] Received Ack
[remote] Packet received: l
[remote] Sending packet: $z0,1b8758b,1#04
[remote] Received Ack
[remote] Packet received: OK
kernel::serialtest::testserial::hex (n=[remote] Sending packet: $m21d27fe40,40#5c
[remote] Sending packet: $m21d27fe40,40#5c
[remote] Sending packet: $m21d27fe40,40#5c
[remote] getpkt: Timed out.
[remote] getpkt: Timed out.
[remote] getpkt: Timed out.
Ignoring packet error, continuing...
<error reading variable: Invalid hex digit 116>) at os/kernel/src/serialtest/testserial.rs:68
68      0..9 => b'0' + n,
(gdb) Quit
(gdb) c
Continuing.
[remote] Sending packet: $Z0,1b8758b,1#e4
[remote] Sending packet: $Z0,1b8758b,1#e4
[remote] Sending packet: $Z0,1b8758b,1#e4
[remote] Sending packet: $Z0,1b8758b,1#e4
[remote] getpkt: Timed out.
[remote] getpkt: Timed out.
[remote] getpkt: Timed out.
Ignoring packet error, continuing...
```

Abbildung 5.2: Timeout

```
(gdb) target remote /dev/ttyUSB0
Remote debugging using /dev/ttyUSB0
[remote] start_remote_1: enter
[remote] Sending packet: $qSupported:multiprocess+;swbreak+;hwbreak+;qRelocInsn+;fork-events+;vfork-events+;exec-events+;vContSupported+;QThreadEvents+;QThreadOptions+;no-resumed+;memory-tagging+;xmlRegisters=i386#72
[remote] Junk: G
[remote] Junk: H
[remote] Junk: =
[remote] Junk: o
[remote] Junk: #
[remote] Junk:

[remote] Junk: 7
[remote] Junk: u
[remote] Junk: `
[remote] Junk: j
[remote] Junk: g
[remote] Junk: {
[remote] Junk: k
[remote] Junk: m
[remote] Junk:

[remote] Junk: n
[remote] Junk: !
[remote] Junk: c
```

Abbildung 5.3: Unexpected Header

5.6 Grenzen der Implementierung

Die aktuelle Implementierung ermöglicht die grundlegenden Debugging-Funktionen im Kernel-Space, besitzt jedoch folgende Einschränkungen:

- Die meisten Funktionen lassen sich nur im Kernel-Space verwenden
- Breakpoints und Step-Into auf Funktionen im Exception-Handler-Pfad funktionieren nicht
- Der Stub muss die einzige Komponente sein, die den seriellen Port nutzt
- Der Stub funktioniert nicht korrekt auf der neusten Version von D3OS. Während er zwar in QEMU funktioniert, sofern der Scheduler auf einen Core eingestellt ist, stürzt das System auf richtiger Hardware noch ab.

Kapitel 6

Fazit und Ausblick

6.1 Fazit

Ziel der Entwicklung des GDB-Stubs für D3OS war die Möglichkeit, das Betriebssystem auf echter Hardware debuggen zu können. Anfangs bot das System nur die Möglichkeit in einer emulierten Umgebung einen Debugger zu verwenden. Durch die Implementierung des Stubs kann der Kernel von D3OS, und in begrenztem Ausmaß auch der User-Space, auf echter Hardware debuggt werden. Der Stub ermöglicht die wichtigsten Funktionen eines Debuggers, wie das Setzen und Entfernen von Breakpoints, auslesen von Speicher und die schrittweise Ausführung des Programms. Wie in der Evaluation aber bereits beschrieben, dürfen aufgrund der Funktionsweise des GDB RSP und der Exception Handler keine Breakpoint oder Step-Into Befehle für bestimmte Funktionsaufrufe verwendet werden. Um bestimmte Funktionen vom Step-Into auszuschließen, lässt sich der `skip` Befehl verwenden. Außerdem ist der Stub noch nicht kompatibel mit Multi-Core-Scheduling und funktioniert auf echter Hardware nur auf einer älteren Version von D3OS aufgrund der in der Evaluation beschriebenen Probleme bei der Datenübertragung. Im Abgaberepository¹ unter der branch `abgabe` zu dieser Arbeit befindet sich die Version des Stubs, die auf Hardware funktioniert und einen alten Stand von D3OS nutzt. Die Entwicklungsversion, die nur auf QEMU zuverlässig funktioniert, aber die neuste Version von D3OS unterstützt, ist unter der branch `development` zu finden.

6.2 Ausblick

Die Implementierung lässt sich in folgenden Bereichen weiterentwickeln:

- **User-Space Kompatibilität:** Aktuell lässt sich der User-Space ausschließlich über Hardware Breakpoints debuggen. Für alle anderen Befehle muss der Adressraum geladen werden, was der Stub noch nicht macht. Um Nutzer-Anwendungen richtig zu debuggen, muss der Stub hier erweitert werden.
- **Neuste Version von D3OS:** Der Stub funktioniert in QEMU mit der neusten Version von D3OS und dem neuen Multi-Core-Scheduler. Dazu muss in der Konfiguration ein Core eingestellt werden. Auf echter Hardware treten aber noch die

¹<https://git.hhu.de/bsinfo/thesis/ba-mof81wap>

in der Evaluation beschriebenen Probleme auf. Der Stub kann hier um Multi-Core-Kompatibilität erweitert werden. Die Probleme des seriellen Ports können ebenfalls weiter untersucht werden.

- **TCP statt Serial:** Alternativ zum seriellen Port kann TCP verwendet werden. Dazu benötigt D3OS Treiber, die eine TCP-Verbindung bei der Verwendung auf echter Hardware zulässt. Sobald eine Verbindung hergestellt werden kann, kann die Implementierung der `GdbConnection` auf TCP geändert werden. Die Implementierung der einzelnen Schnittstellen des Targets und der EventLoop müssen dazu nicht verändert werden.
- **Weitere GDB Feature:** Das GDB RSP und die Crate `gdbstub` bieten noch viele Erweiterungen für verschiedene Anwendungsfälle an. Durch die Nutzung von IDETs der Crate lässt sich die Implementierung leicht um weitere Traits erweitern. Eine sinnvolle Erweiterung wären zum Beispiel Hardware Watchpoints. Dadurch lassen sich Breakpoints auf Speicherzugriffe setzen.

Literatur

- [1] GDB Developers. “GNU Debugger”, Free Software Foundation, besucht am 23. Juni 2026. Adresse: <https://sourceware.org/gdb/>
- [2] Abteilung Betriebssysteme, HHU Düsseldorf. “D3OS Repository”, besucht am 22. Juni 2026. Adresse: <https://github.com/hhu-bsinfo/D3OS>
- [3] QEMU Project Entwickler. “QEMU”, besucht am 1. Aug. 2026. Adresse: <https://www.qemu.org/>
- [4] daniel5151. “gdbstub Crate”, besucht am 23. Juni 2026. Adresse: <https://docs.rs/gdbstub/latest/gdbstub/>
- [5] Microsoft Corporation. “Visual Studio Code”, besucht am 4. Aug. 2026. Adresse: <https://code.visualstudio.com/>
- [6] OpenAI. “ChatGPT”, besucht am 4. Aug. 2026. Adresse: <https://chatgpt.com/>
- [7] C. N. Steve Klabnik und C. Krycho. “The Rust Programming Language”. with contributions from the Rust Community, besucht am 22. Juni 2026. Adresse: <https://doc.rust-lang.org/stable/book/title-page.html>
- [8] Abteilung Betriebssysteme, HHU Düsseldorf. “D3OS Project”, besucht am 22. Juni 2026. Adresse: <https://www.cs.hhu.de/en/research-groups/operating-systems/research/translate-to-english-hermes-1>
- [9] OSDev Wiki Autoren. “Microkernel”, besucht am 23. Juni 2026. Adresse: <https://wiki.osdev.org/Microkernel>
- [10] Abteilung Betriebssysteme, HHU Düsseldorf. “hhuOS”, besucht am 27. Juli 2026. Adresse: <https://www.cs.hhu.de/lehrstuehle-und-arbeitsgruppen/betriebssysteme/forschung/hhuos>
- [11] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer’s Manual, Volume 3: System Programming Guide*, Intel Corporation, Juni 2016.
- [12] daniel5151. “gdbstub_arch Crate”, besucht am 25. Juni 2026. Adresse: https://docs.rs/gdbstub_arch/latest/gdbstub_arch/
- [13] phil-opp und rust-osdev. “x86-64 Crate”, besucht am 4. Aug. 2026. Adresse: https://docs.rs/x86_64/latest/x86_64/

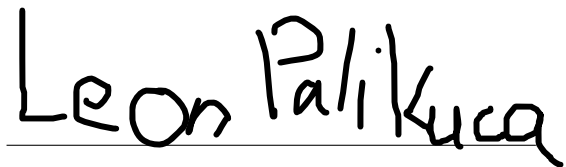
Abbildungsverzeichnis

2.1	Unterschiede Breakpoint Behandlung	6
3.1	Registerwerte auf dem Stack	10
5.1	Stub schickt Pakete zurück	25
5.2	Timeout	25
5.3	Unexpected Header	26

Ehrenwörtliche Erklärung

Hiermit versichere ich, die vorliegende Bachelorarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt zu haben. Alle Stellen, die aus den Quellen entnommen wurden, sind als solche kenntlich gemacht worden. Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Düsseldorf, 4. August 2026


Leon Palikuca